

Framework-Level Accessibility Remediation

*Rebuilding the Foundation:
Turning Accessibility Into Infrastructure*

FOCUS

Framework-Level Accessibility

SCOPE

30+ Components Remediated

YEAR

2024

PORTFOLIO CASE STUDY

TABLE OF CONTENTS

01 Overview / Intro

- When the Problem Was Bigger Than the Screens
 - Hero Metrics
-

02 Empathize

- Experiencing the Product Through Assistive Technology
 - Accessibility Persona: Jordan
 - Atomic Accessibility Research Framework
 - Accessibility Audit Snapshot
-

03 Define

- Identifying the Systemic Root Cause
 - Component Prioritisation Matrix
-

04 Ideate

- Negotiating Between Ideal Accessibility and Technical Reality
 - Engineering Trade-Off Matrix
 - Framework Architecture Diagram
-

05 Prototype

- Demonstrating Accessibility Through Functional Proof
 - Accessibility Design Handoff Checklist
 - Leadership Recognition
-

06 Test

- Embedding Accessibility Into Quality Operations
 - Accessibility QA Framework
-

07 Outcomes & Key Learnings

- Transforming Accessibility From Remediation to Infrastructure
 - Continuous Improvement Loop
 - Key Learnings
-

When the Problem Was Bigger Than the Screens

Recognising an architectural failure, not a surface problem

01

THE STARTING POINT

When I joined the frontend framework initiative, accessibility was not functioning as a layer of quality within the product. It was largely absent from the foundation entirely.

The platform supported a large K–12 ecosystem used daily by students, parents, teachers, and administrators across desktop, mobile, and kiosk experiences. But beneath the visual interface, core accessibility fundamentals were inconsistent or missing altogether.

WHAT WAS BROKEN

Screen Readers

Struggled to interpret navigation — no semantic structure to parse

Form Controls

Lacked accessible names — inputs were invisible to assistive technology

Interactive Elements

Relied heavily on non-semantic HTML — ARIA absent or incorrect

Keyboard Navigation

Users could not consistently move through core workflows

Data Tables

Lacked structure and context — screen readers read as flat text

Validation Messaging

Error messages visually present but inaccessible to AT

THE CRITICAL INSIGHT

At the time, many accessibility conversations focused on fixing individual screens after issues were discovered. Through testing and analysis, I realised the root issue existed much deeper within the frontend framework itself.



This was not a collection of isolated bugs. It was an architectural problem. Continuing screen-by-screen would trap the organisation in an endless remediation cycle.

This project became an opportunity to fundamentally shift how accessibility was implemented across the entire product ecosystem — rebuilding the underlying design and development infrastructure so accessibility could scale systematically.

HERO METRICS

COMPONENTS REMEDIATED

30+

Framework-level fixes with cascading downstream impact

HIGH-IMPACT SCREENS IMPROVED

10

Prioritised by student and parent usage data

PLATFORMS SUPPORTED

Desktop · Mobile · Kiosk

Consistent accessibility across all form factors

CORE FOCUS

Framework-Level Accessibility

Fixing the foundation, not patching the surface

ACCESSIBILITY SCOPE

Inputs · Navigation · Tables · Validation

End-to-end component coverage

Experiencing the Product Through Assistive Technology

Feeling the failure before diagnosing it

02

IMMERSIVE TESTING

Before defining solutions, I immersed myself in the product experience as users with disabilities would encounter it. The goal was not to audit from a distance — it was to feel the friction firsthand.

Screen Reader Testing

NVDA and VoiceOver — evaluating semantic interpretation

Keyboard-Only Navigation

Tab, shift-tab, arrow keys — no mouse at any point

Zoom & Magnification

200–400% zoom — layout, reflow, readability

Contrast Analysis

All interactive states — default, hover, focus, error

Voice Control Evaluation

Command discoverability across navigation and forms

WHAT TESTING REVEALED

Simple workflows became exhausting. Navigation lacked structure. Screen readers announced incomplete or misleading information. Keyboard focus disappeared unpredictably. Error messages were visually present but inaccessible to assistive technology.

The deeper I tested, the more apparent it became that accessibility issues were not isolated defects. They were recurring architectural patterns replicated across the system.

To build shared understanding, I facilitated collaborative testing sessions with engineers and designers — asking them to navigate core workflows using only keyboard input and screen reader feedback. The reaction was immediate: accessibility stopped feeling abstract and became operationally urgent.

ACCESSIBILITY PERSONA

ACCESSIBILITY PERSONA

J

Jordan

High School Student · Age 16

Chromebook · Keyboard-only

Recovering from a wrist fracture, Jordan relies entirely on keyboard navigation to complete daily schoolwork. Accessibility isn't a preference — it's a necessity.

GOALS

- Complete school tasks independently
- Navigate the portal without losing orientation
- Keep pace with peers despite injury

FRUSTRATIONS

- Focus disappears — must restart from the top
- Dropdowns unreachable by keyboard
- Error messages shown but never announced

BEHAVIOURS

- Tabs slowly, listens for focus cues
- Zooms in to identify interactive elements
- Avoids broken areas; asks peers for help

“

If I lose focus on the page, I lose track of where I am completely.”

ACCESSIBILITY AUDIT SNAPSHOT

Dropdown Menus Keyboard inaccessible — no focus entry point Users unable to complete selections independently	Critical
Data Tables Missing semantic headers — no row/column relationships Screen reader reads as flat, unstructured text	High
Validation Errors Errors not announced to assistive technology Users unaware of submission failures or required fields	Critical
Buttons Missing accessible names — icon-only with no label Ambiguous interaction — user cannot predict outcome	High

ATOMIC ACCESSIBILITY RESEARCH FRAMEWORK

Translating Technical Failures Into Product Decisions

Rather than presenting disconnected violations, I structured findings through a layered progression — transforming accessibility from subjective observations into actionable product decisions grounded in user impact.

EXPERIMENT We tested... Keyboard, screen reader, zoom, voice control	FACT We observed... Users unable to navigate dropdowns or complete forms	INSIGHT This suggests... Navigation and form structure lack semantic predictability	RECOMMENDATION We should... Rebuild using semantic HTML and standardised naming patterns
--	--	---	--

REAL RESEARCH EXAMPLES

EXPERIMENT Screen reader testing on forms	FACT Inputs announced without context — 0 accessible names on key form	INSIGHT Users cannot confidently complete workflows or understand field purpose	RECOMMENDATION Standardise accessible naming conventions across all form components
EXPERIMENT Keyboard testing on navigation	FACT Focus order inconsistent — users land on unexpected elements	INSIGHT Users lose spatial orientation and cannot predict navigation flow	RECOMMENDATION Rebuild focus management logic at the framework level
EXPERIMENT Zoom testing at 200%	FACT Layout breaks — content clips or reflows off-screen	INSIGHT Content entirely inaccessible for low-vision users	RECOMMENDATION Refactor responsive structure to support 200–400% zoom

Identifying the Systemic Root Cause

Moving from screen evaluation to framework mapping

03

FROM SCREENS TO SYSTEMS

Once testing patterns became clear, I shifted from evaluating individual screens to mapping the underlying framework architecture. This shift was not academic — it was necessary. Fixing symptoms without addressing structure would guarantee recurrence.

I identified over 30 reusable components requiring remediation, alongside 10 high-impact public-facing screens prioritised based on usage data from students and parents.

COMPONENT CATEGORIES

Inputs & Form Controls

Labels, accessible names, error associations, field groupings

Buttons & Interactive Elements

Accessible names, focus states, pressed/hover states

Navigation Systems

Landmark structure, focus order, skip links, semantic hierarchy

Data Tables

Column/row headers, captions, cell relationships

Validation Patterns

Error announcement, live regions, field association

Structural Hierarchy

Heading levels, landmark regions, reading order

Modal Interactions

Focus trapping, dismiss behavior, return focus

Interaction States

Focus, hover, pressed, disabled — all visible and accessible

CORE FRAMEWORK ISSUES

Non-Semantic HTML

Framework generated div-based UI by default — no native roles

Missing Accessible Names

Components rendered without labels — invisible to screen readers

Inconsistent ARIA Usage

ARIA applied without semantic HTML base — creating false confidence

Poorly Structured Tables

Data relationships absent — screen readers could not interpret grid

Unpredictable Focus Management

Focus order disconnected from visual layout and user expectation



The framework itself was generating inaccessible experiences by default. The goal was no longer remediation — it was changing the foundation that produced accessibility outcomes across the entire product.

COMPONENT PRIORITISATION MATRIX

Inputs & Form Controls

Frequency: Very High · User Impact: Critical



Navigation Systems

Frequency: Very High · User Impact: Critical



Data Tables

Frequency: High · User Impact: High



Modals

Frequency: Medium · User Impact: High



Tooltips

Frequency: Medium · User Impact: Medium



Negotiating Between Ideal Accessibility and Technical Reality

Collaborative problem-solving across design and engineering

04

THE COLLABORATION CHALLENGE

This phase became deeply collaborative. While ideal accessibility solutions often existed theoretically, implementation constraints within the framework required constant negotiation between design intent, engineering feasibility, and product timelines.

I partnered closely with engineers to explore multiple implementation approaches, moving between native HTML replacements, ARIA enhancements, framework-level abstractions, and progressive remediation strategies.

Some engineers initially resisted large-scale framework changes due to the perceived scope and technical debt involved. To build alignment, I focused on demonstrating how systemic fixes would ultimately reduce long-term remediation effort and improve product consistency overall.

ENGINEERING TRADE-OFF MATRIX

Native HTML Replacements

- + Strong, robust accessibility support — screen reader and keyboard native
- Requires full component refactor — higher initial engineering cost

Preferred

ARIA Enhancement

- + Faster initial implementation — lower barrier to adoption
- Less robust — ARIA without semantic base creates fragile patterns

Supplemental only

One-Off Screen Fixes

- + Faster short-term delivery — visible quick wins
- Entirely unsustainable — same issues recur with every new component

Rejected

DESIGN TOKEN STRATEGY

This phase also established design tokens as central to the solution strategy. Rather than manually correcting contrast and interaction inconsistencies across products repeatedly, scalable token systems were developed governing colour contrast, focus states, hover states, pressed states, and interaction visibility.



Accessibility shifted from something manually enforced into something structurally embedded within the design system itself.

FRAMEWORK ARCHITECTURE DIAGRAM



Accessibility decisions made at the token level cascade automatically through components, templates, and product experiences — removing the need for manual enforcement at every layer.

Demonstrating Accessibility Through Functional Proof

Making the invisible visible through working examples

05

BRIDGING INTENT AND IMPLEMENTATION

One persistent challenge throughout the project was translating accessibility recommendations into implementation confidence. Engineers needed to experience correct accessibility behavior directly — not interpret abstract documentation alone.

To bridge this gap, I created functional HTML prototypes demonstrating correct semantic structure, keyboard interaction behavior, screen reader announcements, focus management patterns, and accessible validation behavior.

These prototypes became critical collaboration tools because they removed ambiguity from the implementation conversation entirely.

IMPROVING DESIGN HANDOFF

In parallel, I worked closely with designers to improve accessibility annotation practices during handoff — creating a stronger connection between design intent and implementation quality.

ACCESSIBILITY DESIGN HANDOFF CHECKLIST



Inputs

Accessible labels defined for all form controls



Focus States

Clearly documented for all interactive components



Error Handling

Screen reader behavior and live region specified



Navigation

Keyboard expectations and focus order documented



Interaction States

Hover, pressed, focus, and disabled all included

LEADERSHIP RECOGNITION



Lead-level ownership through systems thinking, accessibility expertise, and cross-project consistency. Contributions extended beyond remediation and into organisational systems thinking — defining accessible interaction state tokens, supporting scalable design system implementation, and driving cross-project accessibility consistency.

— UX Leadership, reflecting cross-functional impact and systems-level accessibility contribution

Embedding Accessibility Into Quality Operations

From ad-hoc checking to structured validation

06

INTEGRATING INTO QA WORKFLOWS

As remediation efforts matured, accessibility validation became integrated directly into QA processes. Previously, accessibility was inconsistently tested or deferred until late-stage review — creating regression risk and rework cycles.

I collaborated with QA teams to establish structured validation criteria and repeatable testing workflows that could run alongside standard sprint ceremonies.

Keyboard Testing

Validate all interactive elements without mouse — every sprint

Screen Reader Validation

Accurate semantic announcements across all component states

Zoom Testing

Functional and readable at 200% magnification

Contrast Verification

WCAG-compliant ratios across default, hover, focus, error states

Focus Management Checks

Visible, logical, predictable focus for all interactions

ACCESSIBILITY QA FRAMEWORK

Keyboard Navigation

Fully operable without mouse — no traps, no lost focus

Pass Criteria

Screen Reader Support

Accurate, complete semantic announcements in all states

Pass Criteria

Focus Visibility

Clear, persistent visual indication — WCAG 2.4.11 compliant

Pass Criteria

Contrast

4.5:1 text, 3:1 UI — verified across all interactive states

Pass Criteria

Zoom Support

Fully functional and legible at 200% magnification

Pass Criteria



Accessibility became integrated into definition-of-done criteria across teams — dramatically reducing recurring regressions and improving long-term sustainability.

OUTCOMES & KEY LEARNINGS

Transforming Accessibility From Remediation to Infrastructure

The foundation became the product

SUMMARY OF IMPACT

By the conclusion of the initiative, the product had evolved from a fragmented accessibility experience into a functional and scalable accessibility foundation. Perhaps most importantly, accessibility stopped being treated as an isolated compliance layer — it became part of the product architecture itself.

30+ FRAMEWORK COMPONENTS REMEDIATED

Cascading downstream improvement across thousands of product screens

10 HIGH-IMPACT SCREENS IMPROVED

Prioritised by real usage data from students and parents

ACCESSIBILITY INTEGRATED INTO QA

Validation criteria embedded in definition of done — every sprint

DESIGN TOKEN INFRASTRUCTURE

Scalable accessibility built into the system — not enforced manually

CROSS-FUNCTIONAL ADOPTION

Design, Engineering, QA, and Product aligned on shared standards

DIRECT USER FEEDBACK LOOPS

Integrated into design evaluation — closing the research-to-delivery gap

CONTINUOUS IMPROVEMENT LOOP



Each cycle of the loop strengthens the foundation — user feedback drives UX evaluation, framework updates are validated through QA, and improvements ship into the live product. The system is self-reinforcing.

KEY LEARNINGS



Fix the factory, not just the output — Screen-by-screen remediation is unsustainable at enterprise scale. The only durable path is changing the system that generates the product.



Make engineers feel the failure — Asking engineers to navigate with keyboard and screen reader changed conversations faster than any audit document could. Empathy is an engineering tool.



Design tokens are an accessibility infrastructure — Embedding contrast, focus, and interaction state requirements into tokens removed the need for manual enforcement at every component level.



Prototypes replace ambiguity — Functional HTML prototypes resolved implementation debates in minutes that documentation had failed to resolve in weeks.



Sustainability requires integration — Accessibility becomes sustainable only when it is embedded into the systems that create products — sprint gates, definition of done, and QA criteria — not applied afterward as correction work.

Accessibility becomes sustainable when it is embedded into the systems that create products — not applied afterward as correction work.
